

ECE 6745 Design Project 3 Report

Complex Digital ASIC Design

1 Introduction

1.1 Motivation and Main Idea

This project builds a dynamic Out-of-Order (OoO) execution core. Our baseline is a 5-stage, fully bypassed, in-order scalar processor based on the TinyRV2 ISA. The bypass network handles simple data hazards, but the pipeline still executes strictly in order. Long-latency instructions, such as our 4-cycle integer multiplier, stall the processor and block all younger instructions, even those that have no dependency on the multiplier. This caps the Instruction-Level Parallelism (ILP) the pipeline can exploit.

We address this by adding an OoO microarchitecture that schedules independent instructions to execute before stalled ones, so the functional units stay busy. We extend the datapath with register renaming to remove false dependencies, an Issue Queue (IQ) for wakeup and selection, and a Reorder Buffer (ROB) for in-order commit and precise exceptions. We pursue this idea because it can hide the latency of complex arithmetic without changing the architectural specification of the ISA.

1.2 Literature Review

We reviewed two superscalar processors that established the modern OoO template, the MIPS R10000 [2] and the Alpha 21264 [1]. Both designs solve the same three problems that arise once instructions are allowed to execute out of order: name-based register hazards, dependency-driven scheduling, and precise exception handling.

To eliminate name-based hazards, both processors use physical register renaming. The R10000 maps 32 logical integer registers into a pool of 64 physical registers through a map table that is updated each cycle as instructions decode [2]. The Alpha 21264 takes the same approach at larger scale, mapping 31 architectural integer registers and 31 floating-point registers into two 80-entry physical register files [1]. In both cases the renaming step removes WAW and WAR hazards by giving every dynamic write its own physical destination, so the scheduler only needs to track true RAW dependencies.

For dynamic scheduling, both designs decouple decode from execute through issue queues. The R10000 splits its window into three 16-entry queues for integer, floating-point, and address operations; instructions wait in their queue until all sources are ready, then issue to a function unit [2]. The Alpha 21264 uses a 20-entry integer queue and a 15-entry floating-point queue, and can issue up to four integer and two floating-point instructions per cycle [1]. The two organizations differ in their partitioning, but both implement the same core idea: hold instructions in a window and let operand readiness, rather than program order, drive issue.

For correctness, both designs commit results in program order even though execution is out of order. The R10000 uses a 32-entry active list that records every in-flight instruction and releases its old physical destination back to the free list when the instruction graduates [2]. The Alpha 21264 uses a larger 80-entry in-flight window with the same role [1]. In-order graduation gives the programmer a sequential view of execution and supports precise exceptions.

Our design adopts the same three mechanisms but at much smaller scale. We use a 64-entry physical register file, a 4-entry IQ, and an 8-entry ROB, with a single ALU and a single 4-cycle multiplier, single-issue throughout. We also leave out two features that both reference designs rely on, speculative execution and branch prediction, because our current scope is limited to arithmetic instructions. The smaller window costs ILP relative to the reference designs, but it keeps the design tractable for a single-semester project and lets us focus on getting the rename, issue, and commit logic correct before scaling.

1.3 Project Overview

We built the OoO processor incrementally on top of the existing in-order datapath. The result is a single-issue OoO processor that covers the arithmetic, CSR, and memory subsets of TinyRV2; only control flow remains as future work. The design includes:

- **OoO execution engine:** A Fetch, Dispatch, Issue, Execute, Commit pipeline with three parallel FU lanes after Issue: a 1-cycle ALU, a 4-cycle pipelined integer multiplier, and a single-in-flight load/store unit.
- **Register renaming and PRF:** A Rename Unit that maps 32 architectural registers to a 64-entry Physical Register File. The PRF has 2 read ports and 3 write ports (ALU, MUL, load) with write-first read forwarding.
- **Issue Queue and Reorder Buffer:** A 4-entry IQ for dynamic wakeup and selection, and an 8-entry ROB for program-order commit and freelist recycling. The IQ enforces in-order issue between memory operations while letting ALU and MUL traffic overtake stalled $1w/sw$.
- **Memory subsystem:** An encapsulated ProcMemunit with a valid/ready interface to the IQ. A load completion drives a third PRF/ROB writeback port and broadcasts on the IQ wakeup network.

Control flow is out of scope for this project: branches, jumps, and the `lui/auipc` instructions are not implemented. As future work, we plan basic in-order branch resolution at the ROB head, which would keep recovery simple and avoid speculative rollback.

The processor passes the full reused ECE 4750/5740 `rr`, `rimm`, `csr`, and `mem` test suites, the seventeen OoO-specific directed tests of Section 4.3, and the 240 self-checking differential random tests of Section 4.4.

1.4 Overview of Results

At a common 10 ns clock period, the OoO processor passes the full ASIC flow with positive setup and hold slack and no DRC, antenna, or LVS violations. Functional correctness is verified at four levels (PyMTL RTL simulation, VCS RTL simulation, fast-functional gate-level simulation, and back-annotated gate-level simulation), each running the full reused test suite, the seventeen OoO-specific directed tests of Section 4.3, and the 240 self-checking differential random tests of Section 4.4. Integrated with SRAM-based instruction and data caches into a single-core system, the design closes timing cleanly at the same 10 ns target. Notably, the in-order baseline system fails to close 10 ns (-0.0053 ns STA setup slack) at the same constraint: the OoO pipeline’s issue-queue stage breaks the ALU-to-cache combinational path that becomes the SysBase critical path, so the more complex OoO system is in fact easier to close than the simpler baseline.

At the RTL level, OoO execution gives a $1.51\times$ standalone / $1.44\times$ system speedup on `vmuladd2-unopt`, a $1.24\times$ / $1.16\times$ speedup on `dotprod-unopt-unrolled`, and essentially no measurable speedup ($\sim 1.01\times$) on `chacha20-unrolled` and `vvadd-sw-opt`. The pattern matches the design intent: dynamic scheduling and the pipelined multiplier pay off when the program contains long-latency mul-RAW chains (the two `-unopt` kernels) but yield little on already-tightly-scheduled arithmetic. Section 5.3 attributes the speedup on the mul-heavy benchmarks to a combination of pipelined-multiplier throughput and OoO latency-hiding, since the two designs use different multiplier microarchitectures (baseline iterative vs. OoO pipelined).

The post-PNR breakdown of the isolated processor points at two takeaways. First, the physical register file dominates area (62.13%) and dynamic activity, so the PRF is the primary scalability bottleneck of this microarchitecture, not the rename or commit logic. Second, the critical path lies inside the ALU combinational logic (84.36% of the data arrival time) rather than on any IQ, rename, or ROB path, which confirms the Phase 5 decision to merge Register Read into Issue: the OoO control structures are not the frequency-limiting elements. At the system level, the picture inverts. The I- and D-caches together take 79.7% of the area and a majority of the dynamic power, and the processor core becomes a minority contributor. Cache miss penalty drives system CPI: a D-cache miss rate of 0.75 on `vmuladd2-unopt` yields system CPIs of 4.39 (SysBase) and 3.05 (SysOoO), well above their standalone CPIs of 1.59 and 1.05. Despite OoO’s higher absolute power, the combination of cycle savings and cache-dominated system power produces a near-energy-neutral result: SysOoO consumes 1–2% less energy than SysBase on three of the four benchmarks. The OoO mechanisms are correct and quantitatively beneficial where ILP exists, are not the timing or area bottleneck of the core, and become essentially free in system energy once caches are integrated. The next gains would come from PRF reduction and from a memory pipeline that can hide cache miss latency.

1.5 Acknowledgements

We thank Professor Christopher Batten for weekly guidance, design discussions, and the course material that shaped this project. We thank Edwin Chen, Kian Mahmoodi, and Wei Zheng of the superscalar

in-order group (Group 6) for sharing their random instruction-mix test generator, which we adapted into our differential regression suite (Section 4.4), and the post-PNR cycle/area/power data of their dual-issue processor used in our cross-design comparison (Section 5.4). We thank the staff of ECE 6745 for the comprehensive lab assignments and tutorials that built up the ASIC design and EDA flow skills this project relies on, and the staff of ECE 4750/5740 for the baseline in-order processor we built directly upon.

2 Baseline Design

Our baseline is a 5-stage, fully bypassed, in-order scalar processor implementing the TinyRV2 ISA. It was designed and verified in the ECE 4750/5740 Computer Architecture labs. Because the baseline is already an optimized in-order pipeline, it is a meaningful reference point for measuring what OoO execution adds.

As shown in Figure 1, the baseline uses the standard five stages: Fetch (F), Decode (D), Execute (X), Memory (M), and Writeback (W).

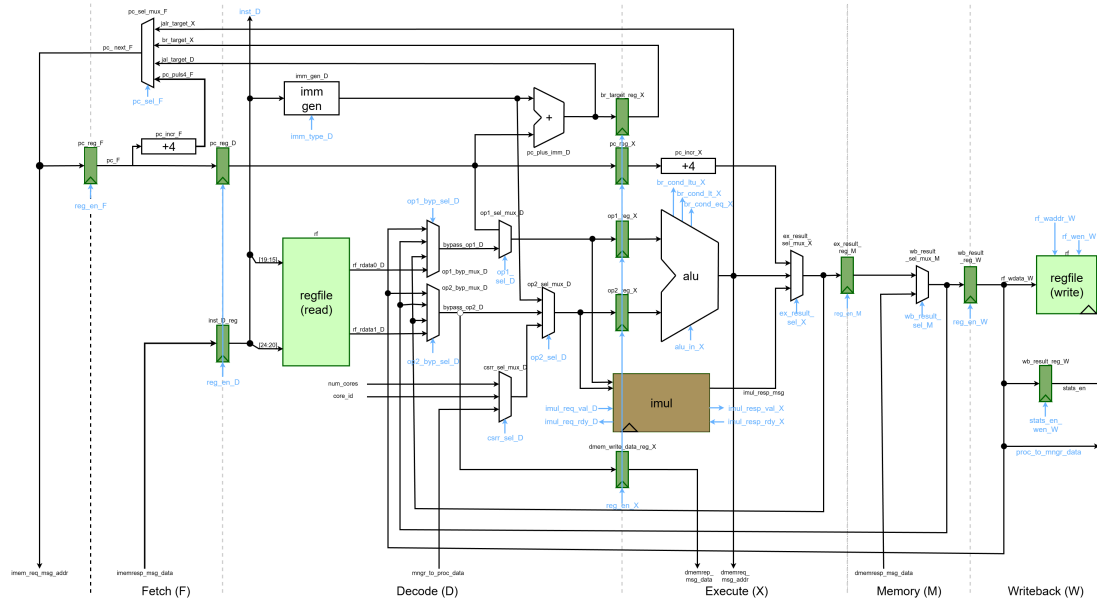


Figure 1: Datapath of the baseline 5-stage fully bypassed in-order processor.

Pipeline behavior. Fetch retrieves an instruction and computes the next program counter; for branches and jumps it predicts $PC+4$ and resolves the actual target in Execute. Decode reads operands from the register file. Execute holds a single-cycle ALU and an iterative 4-cycle integer multiplier (`imul`) that occupies the X stage from issue to completion and cannot accept a new `mul` until the current one finishes. The OoO design in Section 3 replaces this with a pipelined multiplier of the same 4-cycle latency; Section 5.3 discusses the performance implications of this microarchitectural difference. Memory handles loads and stores, and Writeback returns results to the register file.

Bypass network. To reduce stalls from RAW hazards, the baseline forwards results from the ends of X, M, and W back to the operand multiplexers in D. This lets dependent instructions execute back to back in most cases. The remaining structural exception is the load-use hazard, which still needs a one-cycle stall.

Why this is a good starting point. A fully bypassed in-order processor sets a strict ceiling for single-issue in-order execution. Comparing OoO against this stricter ceiling makes any CPI improvement clearly attributable to dynamic scheduling rather than to forwarding tricks. It also gives a clean reference for the cost side, since any added cycle time, area, or energy from our IQ, ROB, and rename logic shows up directly against an already-optimized in-order core.

The bottleneck OoO targets. The motivating limitation of the baseline is that the iterative multiplier blocks the X stage for several cycles. While `imul` executes, every younger instruction stalls in D, even those with no dependency on the multiplier result. This is the case our OoO design is built to remove, and Section 3 walks through the microarchitectural changes that do so.

3 Alternative Design

A fully functional OoO processor has many interacting parts. To keep the work tractable and each piece independently testable, we built the design over six phases, adding one mechanism at a time and verifying it before moving on.

3.1 Phase 1: Decoupling Execution Latencies

Phase 1 modified the baseline pipeline to support variable-latency execution. We inserted pipeline registers to mark a new Issue (I) stage, though the dynamic issue logic itself was not yet in place. In Execute, we added a fixed 4-cycle pipelined integer multiplier alongside the single-cycle ALU. We removed the Memory (M) stage at this point to keep the focus on execution latency. Figure 2 shows the resulting datapath.

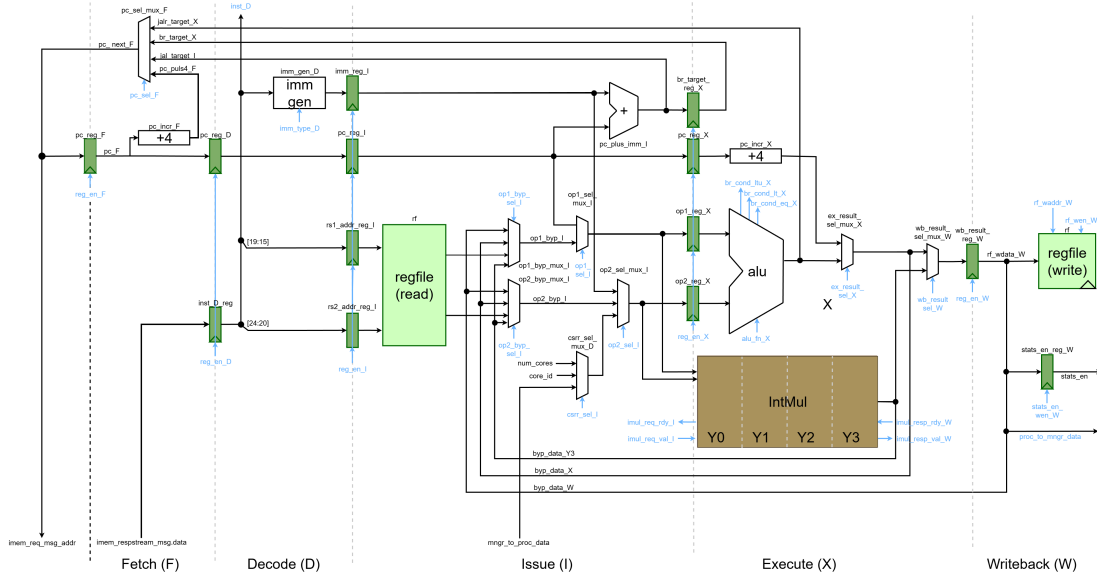


Figure 2: Phase 1 datapath: an Issue stage and a 4-cycle pipelined integer multiplier are introduced.

The goal was to separate the ALU and multiplier latencies in the datapath. With a 1-cycle ALU and a 4-cycle pipelined multiplier as two parallel FU lanes after Issue, instructions can in principle complete out of program order. We kept the baseline bypass network and control logic so we could verify the variable-latency datapath against the existing directed tests before changing anything else.

Pipeline now: $F \rightarrow D \rightarrow I \rightarrow \{ALU \mid IntMul\} \rightarrow W$.

3.2 Phase 2: Stripping Down for Core Verification

After our first review with Professor Batten, we judged that the full bypass network and control flow paths were making the emerging OoO logic harder to debug. We stripped the datapath down to the smallest design that could exhibit out-of-order writeback (Figure 3).

We removed the Writeback (W) stage and put write-first behavior directly in the register file. We removed the bypass network entirely. The supported instruction set was narrowed to `add`, `addi`, and `mul`, and all control instructions were taken out.

These simplifications were deliberate. Without forwarding, RAW hazards must be resolved through stalling alone, which let us verify that dependency tracking was correct before reintroducing forwarding. The `add/addi/mul` subset is the minimum mix that lets us observe a younger `add` writing back before an older multi-cycle `mul`. Folding commit into the register file narrowed write-port arbitration to the ALU and multiplier alone. After validating in-order issue, out-of-order writeback, and RAW stalling in this isolated environment, we were ready to add the issue queue.

Pipeline now: $F \rightarrow D \rightarrow I \rightarrow \{ALU \mid IntMul\}$ (writeback folded into register file).

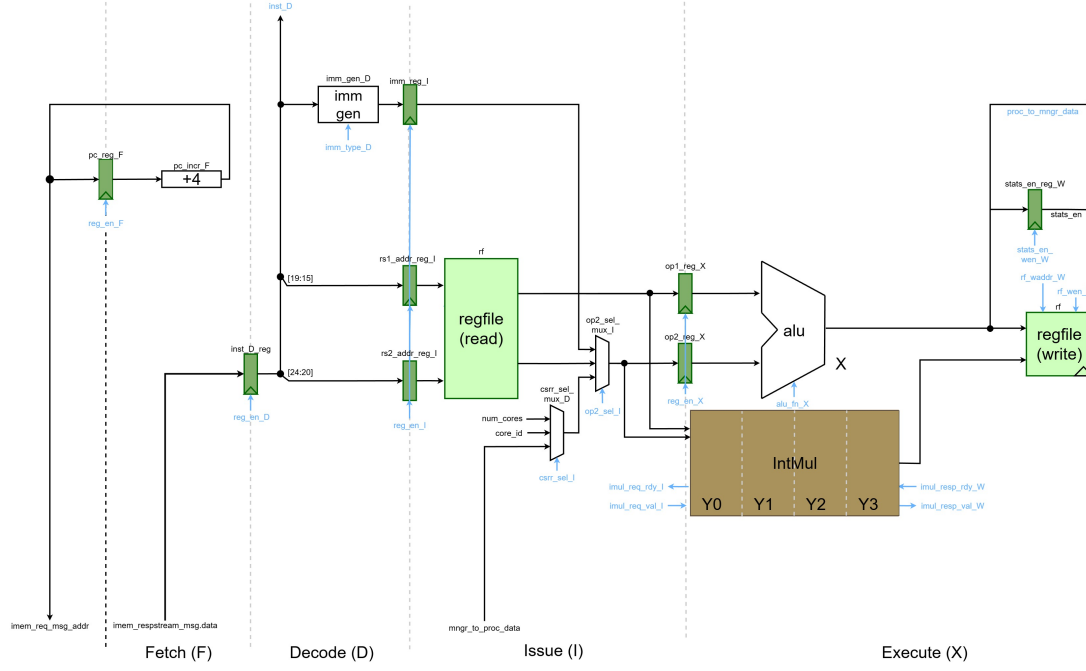


Figure 3: Phase 2 datapath: a stripped-down architecture supporting only `add`, `addi`, and `mul` with no bypass network.

3.3 Phase 3: Issue Queue and Pre-Decoding

Phase 3 added dynamic scheduling. We inserted an Issue Queue (`ProcIssueQueue`) after Fetch. The IQ accepts instructions from Dispatch in program order and issues them out of order once their sources (`rs1`, `rs2`) are ready and no RAW hazards remain.

To keep the IQ's hazard tracking efficient, we added a lightweight Pre-Decode module (`ProcPreDecode`) at the start of Dispatch. It extracts `rd`, `rs1`, and `rs2` with their valid bits and forwards this metadata to the IQ and the rename logic, so the IQ does not redo full instruction decoding.

We kept an explicit Register Read (RR) stage between Issue and Execute as a conservative timing decision. We were worried that collapsing IQ traversal, control signal generation, and register file reads into one cycle could create a long combinational path from the IQ output to the Execute operand registers. A separate RR stage gave us margin until we could measure the path with the EDA flow. At this point the design supported true out-of-order issue and execution of `add`, `addi`, and `mul`, but did not yet handle WAW or WAR hazards.

Pipeline now: $F \rightarrow D \rightarrow I \rightarrow RR \rightarrow \{ALU \mid IntMul\}$.

3.4 Phase 4: In-Order Commit and Register Renaming

To support precise exceptions and remove name-based hazards (WAW and WAR), we added in-order commit and register renaming.

We first integrated a Reorder Buffer (`ProcReorderBuffer`). The ROB allocates an entry for each instruction at Dispatch in program order. In an early iteration, before the PRF was fully independent, we let the ROB temporarily hold execution results and forward them to the Architectural Register File (ARF) at commit. Once full renaming was in place, this data path turned out to be redundant: the ARF only needs mapping updates at commit, not the result data itself.

We then added a Rename Unit. During Dispatch it maps architectural register addresses to physical register addresses and provides the renamed tags to both the IQ (for wakeup tracking) and the ROB (for freelist bookkeeping). On commit, the ROB returns the displaced physical register to the Rename Unit's freelist for reuse. CSR instructions (`csrr`, `csrw`) are marked at Pre-Decode and treated as an IQ-head barrier: a CSR may only issue from the IQ head, and no younger instruction can overtake it while it waits. This preserves the architectural visibility of CSR state without modeling CSR dependencies in the IQ.

With the ROB and Rename Unit in place, the processor reached true OoO issue with in-order commit and correctly handled all three hazard types. Figure 4 shows the architecture at this stage, with the now-redundant RR stage and ARF still present.

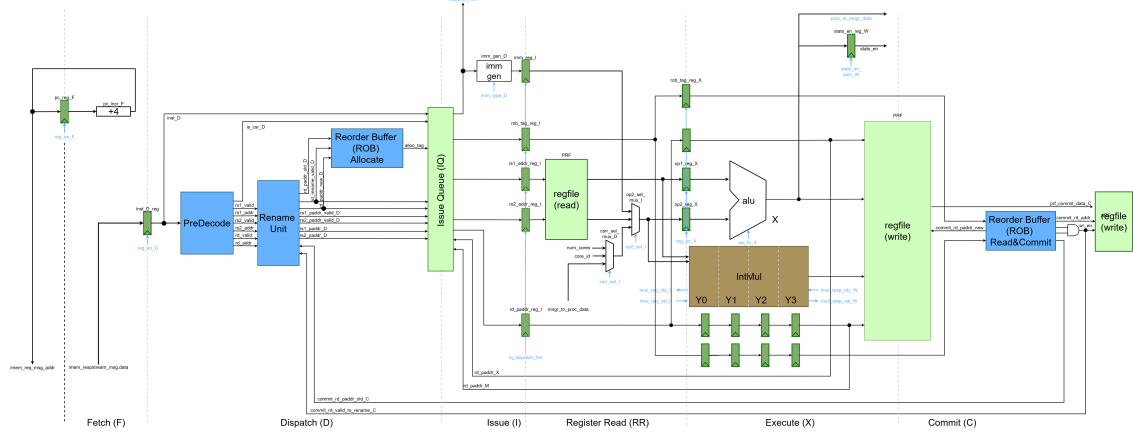


Figure 4: Phase 4 architecture with IQ, ROB, and Rename Unit integrated. The RR stage and ARF are still present.

Pipeline now: $F \rightarrow D \rightarrow I \rightarrow RR \rightarrow \{ALU \mid IntMul\} \rightarrow C$.

3.5 Phase 5: PPA-Driven Optimization

After our second review with Professor Batten, we ran the design through the industrial EDA flow for a preliminary PPA evaluation. The synthesis results pointed at two simplifications.

Removing the RR stage. Initial post-Place-and-Route (PnR) timing reports placed the critical path inside Execute, in the arithmetic logic itself, not on the IQ-to-operand path we had worried about in Phase 3. The dedicated RR stage was no longer earning its keep. Merging Issue and RR into one stage could in principle lengthen that combinational path, so we ran a second PnR iteration with the merged version to verify. Timing reports still placed the critical path in Execute, confirming the merge was safe. The condensed pipeline reduced area, register count, and pipeline-register switching energy without hurting clock frequency.

Removing the ARF. The synthesis tool optimized the ARF away, because every read in our renamed design comes from the PRF and the ARF was only written at commit and never read. We removed the ARF logic from the RTL to keep the source consistent with the synthesized hardware, and refined the PRF into a 64-entry, 2-Read / 2-Write structure (one read port per source operand, two write ports for the ALU and multiplier).

Write-first PRF bypass. We added a write-first read forwarding path inside `ProcPregfile`. When a value is written back in the same cycle that a dependent instruction reads it, the read port returns the just-written value, removing one bubble from back-to-back RAW chains. The optimized final datapath is shown in Figure 5.

Pipeline now: $F \rightarrow D \rightarrow I \rightarrow \{ALU \mid IntMul\} \rightarrow C$.

3.6 Phase 6: Memory Subsystem

Phase 6 added a memory subsystem supporting `lw` and `sw` on top of the renamed pipeline. The memory logic is encapsulated in a self-contained `ProcMemunit` that sits as a third FU lane alongside the ALU and `IntMul`, and three structures (PRF, ROB, IQ) are extended to handle a new variable-latency writeback event. Figure 6 shows the resulting datapath.

MemUnit. A single `busy_M` flag tracks whether a memory instruction is in flight, along with its renamed destination, ROB tag, and `is_sw` bit. The address is computed combinatorially on issue from the `rs1` PRF read and the immediate, and the `dmem` request fires in the same cycle. The unit accepts a new instruction only when it is not busy (or is completing a response in the same cycle) and the `dmem` request port is ready,

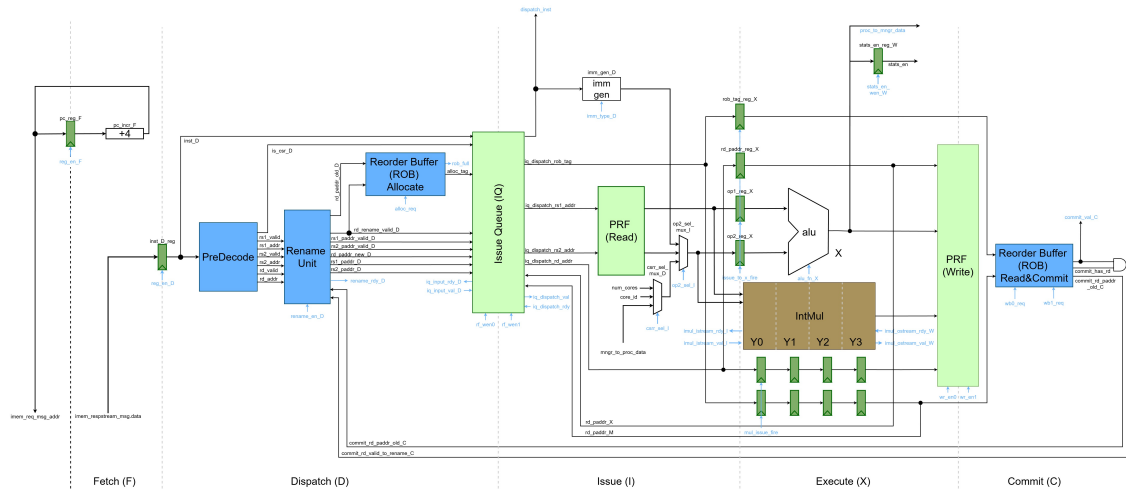


Figure 5: Phase 5 final optimized arithmetic-only datapath: RR and ARF removed, PRF refactored into a 64-entry 2R/2W structure (rs1/rs2 reads; ALU/MUL writes) with write-first read forwarding.

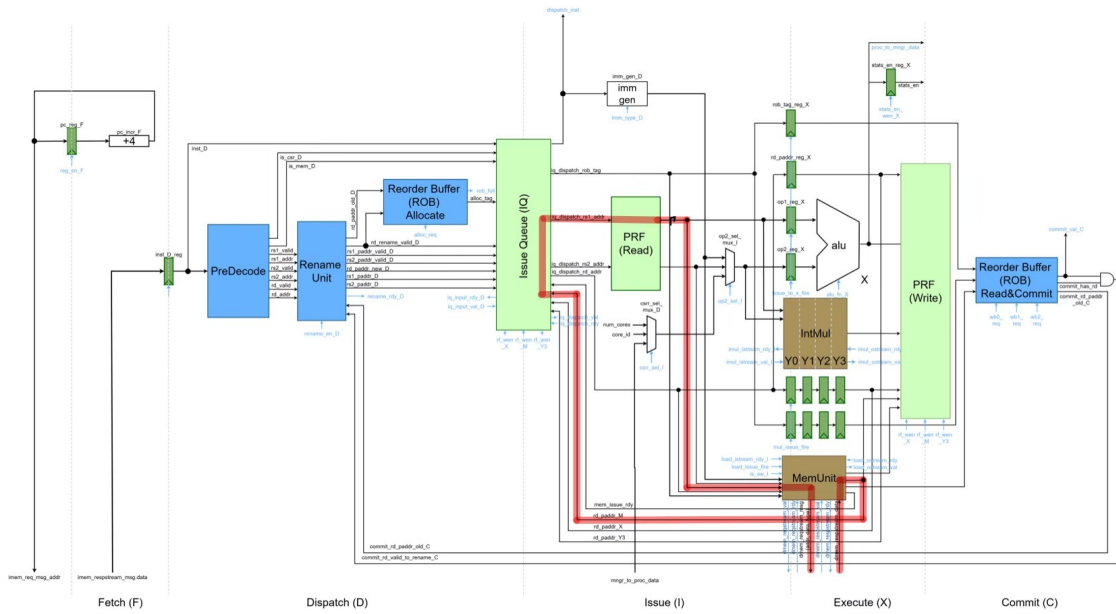


Figure 6: Phase 6 datapath: ProcMemunit added as a third FU lane with a valid/ready interface and `mem_issue_rdy` backpressure to the IQ; PRF and ROB extended with a third writeback port for load completions.

exposing this readiness to the IQ via a new `mem_issue_rdy` signal. This single-in-flight policy avoids memory disambiguation entirely, and the valid/ready interface makes a future LSQ a drop-in replacement.

Memory ordering at the IQ. Memory operations cannot be freely reordered without a disambiguation mechanism, but ALU and MUL operations have no such constraint. The IQ's selection logic encodes this asymmetry directly. When scanning entries from oldest to youngest, the first `is_mem` entry becomes a barrier for younger memory operations: only that oldest `lw/sw` may issue, and only when MemUnit asserts `mem_issue_rdy`. Younger non-memory instructions are still free to issue around it. This preserves program order among memory accesses while letting independent ALU/MUL work overlap with a stalled memory operation.

A third writeback path. A load completion is a new writeback event with arbitrary latency, since the dmem response can arrive many cycles after the request. We extended three structures to handle it:

- **PRF: 2R/2W $\rightarrow$ 2R/3W.** A third write port (`wr_en2`, `wr_addr2`, `wr_data2`) carries the load response data, and the write-first bypass at the read ports is extended to cover it.
- **ROB: third writeback tag port.** A new `wb2_req/wb2_tag` port lets MemUnit clear the pending bit of a completing `lw` or `sw` independently of the ALU and MUL writebacks.
- **IQ scoreboard bypass: third wakeup port.** The IQ’s wakeup logic now compares each waiting entry’s source operands against three concurrent writeback addresses (`rf_wen0/1/2`). When a load response arrives, any IQ entry whose source matches the load’s renamed destination becomes ready in the same cycle and can issue on the next.

Loads therefore behave like long-latency producers in the renaming and wakeup picture: their writeback may arrive many cycles after issue, but when it does, dependent consumers wake up in the same cycle the load forwards its result.

Cycle-level behavior. Figure 7 illustrates the resulting behavior on a representative four-instruction sequence that exercises all three FU lanes. The 4-cycle `mul` (a) occupies the MUL lane across cycles 4–7, while an independent `add` (b) and `lw` (c) execute concurrently on the ALU and Mem lanes (X at cycle 5 and M at cycle 6 respectively). The younger `add` writes back at cycle 5, several cycles before the older `mul`’s Y3 at cycle 7 – out-of-order writeback in action. The dependent `add` (d) sits in the IQ as a single `i` cycle waiting on x3; when `mul` completes Y3 and broadcasts wakeup at cycle 7, d issues in the same cycle (I=7) through the `wb1` bypass network. Despite this reordered execution, commit retires the four instructions in program order at cycles 8, 9, 10, and 11. Trailing `r` cycles for b, c, and d represent the PRF wait while older entries drain through the single-wide commit port. Section 4.4 verifies each of these mechanisms (OoO issue, multi-lane writeback, RAW wakeup, in-order commit) in isolation using paired pipeline diagrams and RTL linetraces from directed tests.

Instruction	1	2	3	4	5	6	7	8	9	10	11
a: <code>mul x3, x1, x2</code> (MUL lane)	F	D	I	Y0	Y1	Y2	Y3	C			
b: <code>add x4, x5, x6</code> (ALU lane, indep.)		F	D	I	X	r	r	r	C		
c: <code>lw x7, 0(x8)</code> (Mem lane, indep.)			F	D	I	M	r	r	r	C	
d: <code>add x9, x3, x4</code> (depends on a, b)				F	D	i	I	X	r	r	C

Figure 7: Representative cycle-level behavior of the Phase 6 OoO pipeline on a synthetic four-instruction sequence exercising all three FU lanes. Independent operations on different lanes execute concurrently; younger instructions write back ahead of older ones; dependent consumers wake up the same cycle their producer’s writeback broadcasts. Commit nonetheless retires all four instructions in program order. Detailed verification of each mechanism appears in Section 4.3.

Pipeline now: $F \rightarrow D \rightarrow I \rightarrow \{ALU \mid IntMul \mid MemUnit\} \rightarrow C$.

4 Testing Strategy

4.1 Methodology Overview

Verifying an out-of-order processor is harder than verifying an in-order pipeline. An OoO design admits many valid micro-architectural traces for the same program, so a test that only checks committed values can pass while hiding bugs in renaming, wakeup, or freelist logic. We organized our tests along four axes: **whitebox unit tests** on each new module, **blackbox directed assembly tests** (both reused and newly written), **random instruction-mix tests** that differentially compare ProcOoO against the ProcFL functional-level reference, and **delay-injected reruns** of all of the above with randomized imem and dmem stalls. Each layer fails differently: a unit-test failure points at one module, a directed-test failure at an interaction, a random-test failure at an unseen instruction sequence, and a delay-test failure at a backpressure path. Together they localize bugs faster than any single layer would.

We redesigned the linetrace so each instruction’s full lifetime fits on one line: the disassembled instruction and ROB tag at IQ insert; the renamed source and destination physical addresses at dispatch; the ALU, MUL, and LW writeback addresses (`alu:p??`, `mul:p??`, `lw:p??`) in the WB column; and the ROB tag, architectural register, and freed old physical register at commit. With the ROB tag visible at both issue and commit, in-order commit is directly observable from the trace.

4.2 Whitebox Unit Tests

For each new module we wrote a `run.test.vector.sim` unit test that drives the ports directly. This reaches states that are hard to trigger from a full assembly program. For example, `rob_full` needs nine in-flight instructions with controlled writeback timing, which we can set up in a few cycles at the unit level. Table 1 summarizes these tests.

Module	Test cases	Key behaviors verified
ProcPreDecode	R / I / S+B / U+J types; CSR; LW/SW; class exclusivity	<code>rs1.valid</code> , <code>rs2.valid</code> , <code>rd.valid</code> per opcode; <code>is_csr</code> and <code>is_mem</code> from disjoint opcode buckets.
ProcRenameUnit	basic rename + commit reclaim	RAT lookup; freelist allocation order; x0 non-rename; <code>paddr</code> recycle on commit.
ProcPregfile	2R/3W; same-cycle bypass; p0 protection across all three write ports; port-2 (load) writeback and bypass; triple write same cycle	Same-cycle write-to-read forwarding; p0 stays 0 under writes from any port; load response visible through bypass on the same cycle it writes.
ProcIssueQueue	independent dispatch; RAW wakeup; mem dispatch; mem blocked by MemUnit busy; mem in-order; ALU overtakes stalled mem; wb2 wakeup	Scoreboard busy/ready tracking; combinational wakeup on three writeback ports; <code>mem_issue_rdy</code> gate; older mem issues before younger mem; younger ALU/MUL bypass a stalled mem.
ProcReorderBuffer	in-order commit; OoO writeback; ROB-full + wrap-around; lw / sw via wb2; three writeback ports same cycle	Tag allocation; in-order head pointer; full signal; freed- <code>paddr</code> reporting; SW commits with <code>has_rd=0</code> ; concurrent wb0/wb1/wb2 do not interfere.
ProcMemunit	LW end-to-end; SW end-to-end; dmem-rdy backpressure; busy blocks new issue; back-to-back issue (req_fire and resp_fire same cycle)	Address = base + imm; store path forwards <code>sw_data</code> and keeps <code>rf_wen=0</code> ; single-in-flight policy; correct metadata handover when an old response and a new request fire on the same cycle.

Table 1: Module-level whitebox unit tests. Each row is a dedicated PyMTL3 test driven through the module’s ports.

Two unit tests stand out. The ROB out-of-order writeback case allocates two entries (tag 0, tag 1), forces a writeback for tag 1 *before* tag 0, and checks that commit waits until tag 0’s writeback arrives before retiring either entry. A processor-level test would only see x2 and x3 receive the right values; the unit test makes commit ordering itself observable, and fails loudly if a younger entry retires ahead of an older pending one.

The IQ ALU `passes a stalled mem` test pins down the asymmetric memory ordering rule. We insert an older `lw` while `mem_issue_rdy` is held low, then insert a younger ALU op behind it. The ALU op must dispatch in the cycle both entries are valid, even though the older mem entry sits at IQ slot 0 and is not yet allowed to issue. A linetrace alone cannot distinguish this from generic OoO issue; the unit test verifies the exact dispatch logic.

4.3 New OoO-Specific Directed Tests

The reused suite verifies single-instruction correctness but does not target the new structures: register renaming under WAW/WAR pressure, the IQ’s out-of-order selection, multi-port writeback, in-order commit, and freelist recycling. We wrote seventeen directed tests in `inst_0o0.py`, each crafted so a particular micro-architectural event is provable from the linetrace, and each pinning down a property no other test covers.

Nine target the arithmetic OoO core; eight target the memory subsystem and its integration with the rest of the pipeline. All tests pass on our final RTL. Table 2 lists them.

Test	Targeted behavior
<i>Arithmetic OoO core</i>	
overtake	Independent ALU ops issue ahead of a stalled mul -dependent inst.
rob_full	Nine in-flight insts force the 8-entry ROB to fill; dispatch stalls.
iq_full	Four dependents on a slow mul fill the 4-entry IQ <i>before</i> the ROB is full.
waw	WAW on x1 : younger writer must not corrupt older write at commit.
war	WAR on x1 : older reader must see the old value despite the younger writer.
waw_war_complex	Dense interleaved WAW/WAR on multiple architectural registers.
x0_penetration	A slow mul x0 , ... must not stall dependents that read x0 .
back_to_back_bypass	Tight dependency chain forces single-cycle wakeup-and-issue every cycle.
freelist_wrap	Seventy unique writes exceed 64 physical registers, exercising freelist recycling.
<i>Memory subsystem and integration</i>	
tri_wb	Three writeback ports (ALU/MUL/LW) target the PRF, ROB, and IQ wakeup network in the same cycle.
mem_bypass_arith	Independent ALU ops issue past a sw stalled in the IQ on dmem.
arith_bypass_mem	An independent lw issues past an add trapped behind a slow mul .
mem_order	A pair of sws and a pair of lws to the same addresses must respect program order.
lw_dependent	A consumer of a lw result waits for wb2 wakeup; an independent ALU op overtakes the dependent.
lw_base_dep	A lw whose base address comes from a slow mul must hold in the IQ until the mul writes back.
sw_data_dep	A sw whose store data comes from a slow mul must hold in the IQ until the mul writes back (rs2 wakeup for mem).
mem_inflight	Burst of four sws plus four lws forces the single-in-flight mem pipe to serialize while ALU continues.

Table 2: Directed assembly tests in `inst.OoO.py`.

A few of these deserve a closer look. **rob_full** and **iq_full** are companion tests for the two backpressure conditions in our design. The ROB has eight entries while the IQ has only four, so a chain of dependents on a slow producer can fill the IQ *before* the ROB is full. We added **iq_full** because a designer who only sized backpressure against the ROB would silently drop entries here. **x0_penetration** writes to **x0** with a 4-cycle **mul** and immediately reads **x0** from a dependent **add**. If rename allocated a real physical register for **x0**, the **add** would stall for four cycles; because **x0** is hardwired to **p0** the **add** dispatches right away. **freelist_wrap** loops 70 **addis** writing to the same architectural register, forcing 70 distinct physical-register allocations and 69 freelist returns. If commit ever fails to return a freed paddr, the program deadlocks at dispatch. We keep **waw** and **war** as standalone two-instruction tests even though **waw_war_complex** subsumes them, because a failure in the dense test is hard to bisect; the isolated pair pinpoints which mechanism (younger-writer renaming vs. older-reader shielding) actually broke.

The memory tests follow the same one-property-per-test logic. **mem_bypass_arith** and **arith_bypass_mem** are duals: the first checks that an ALU op can issue while a mem instruction is held in the IQ on **mem.issue_rdy**; the second checks the reverse, that a **lw** can issue while an **add** is held on a slow **mul**. Together they pin down the asymmetric IQ ordering of Section 3.6, where mem is in-order with respect to mem but not with respect to arithmetic. **lw_base_dep** and **sw_data_dep** close the wakeup loop on mem operands: a mem entry whose rs1 (base for **lw**) or rs2 (data for **sw**) is busy must wait in the IQ exactly like a non-mem entry, ruling out a class of bugs where mem entries get a special-cased wakeup path. **tri_wb** schedules a **mul**, an **add**, and a **lw** so their writeback cycles can overlap, exercising **wr_en0/1/2**, **wb0/1/2**,

and `rf_wen0/1/2` concurrently from real assembly rather than from forced unit-level inputs. This single test strictly subsumes the older ALU+MUL-only dual-writeback scenario, so we did not keep a separate dual-writeback test. `mem_inflight` runs four back-to-back stores followed by four loads; with `delays=True` the dmem path stalls randomly, exercising the `mem_issue_rdy` backpressure across many cycles.

Three tests are illustrated by paired *pipeline diagrams* and *linetraces*. The pipeline diagram lays out each instruction's stages cycle by cycle (F, D, I, X or Y0–Y3 or M, C; lowercase `i` for an IQ-wait cycle and lowercase `r` for a PRF-wait cycle before in-order commit) and is the cleanest view of what the design is supposed to do. The linetrace is the corresponding RTL output and is the evidence that the diagram is what actually happens. Reading the two together makes the OoO behavior provable: every box in the diagram corresponds to a column in the trace.

The `overtake` test (Figures 8 and 9) shows the core OoO behavior end to end. A `mul` at ROB tag 5 produces `p03`; a dependent `add` at tag 6 needs that result; an independent `add` at tag 7 and a `sub` at tag 0 follow in program order. The diagram makes the `overtake` immediately visible: tag 6 (`add x4`) sits as `i i i` for three cycles in the IQ while tag 7 (`add x6`) and tag 0 (`sub x9`) progress through `D` → `I` → `X` two and one cycle ahead of it respectively. The linetrace corroborates this on the RTL: the IS column shows tag 7 and tag 0 issuing several cycles before tag 6, and the WB column then prints `alu:p08 mul:p03` on a single cycle, the dual-writeback event in which the `sub` (tag 0) and the `mul` (tag 5) finish together. In the same cycle, the dependent `add` (tag 6) issues, woken combinationally by `mul:p03` on the bypass network. Commit then retires tags 5, 6, 7, 0 in program order, even though tag 7 wrote back several cycles before tag 6, visible in the diagram as the trailing `r r r` sequences for tags 7 and 0. One pair of figures covers OoO issue, dual writeback, same-cycle wakeup, and in-order commit.

Instruction	1	2	3	4	5	6	7	8	9	10	11
a: mul x1,x2,x3	F	D	I	Y0	Y1	Y2	Y3	C			
b: add x4,x1,x5		F	D	i	i	i	I	X	C		
c: add x6,x7,x8			F	D	I	X	r	r	r	C	
d: sub x9,x8,x7				F	D	I	X	r	r	r	C

Figure 8: `overtake` pipeline diagram. Tag 7 (`add x6`) and tag 0 (`sub x9`) reach `X` at cycles 6 and 7, while tag 6 (`add x4`), which depends on tag 5's `mul` result, sits as `i i i` until cycle 7 and only reaches `X` at cycle 8. Trailing `r` cycles for tags 7 and 0 are PRF waits before in-order commit.

The `waw_war_complex` test (Figures 10 and 11) shows renaming and freelist return at full pressure. The diagram packs nine instructions over sixteen cycles and makes the WAW/WAR resolution visible at a glance: instruction `c` (`add x3,x4,x1`, the WAW on `x3`) reaches `X` at cycle 6, one cycle *before* the older instruction `a` (`mul x3,x1,x2`) writes back at cycle 7. Younger and older write the same architectural register in inverted order, but rename has assigned them different physical registers (`p09` and `p04`), so neither stomps on the other. Instruction `e` (`add x11,x4,x1`, reads *old* `x4`) reaches `X` at cycle 8, one cycle before instruction `f` (`add x4,x9,x10`, the WAR writer) reaches `X` at cycle 9; in the diagram, the older reader's `X` sits left of the younger writer's `X` on the same architectural register, which is the WAR-by-rename property made visual. Instruction `h` (`mul x6,x3,x2`) issues against the new `x3` (`p09`, from `c`, not `p04` from `a`), and the linetrace confirms: the IS column prints `5:mul p03,p09,p33`, picking up the post-WAW mapping. The Commit column shows `x03` retiring multiple times in program order, each commit freeing a different old paddr (`free:p03`, `free:p04`, `free:p09`, ...). Late in the trace, `mul:p03` reappears in the WB column: the freelist has recycled `p03` into a later allocation, closing the allocate-free-allocate loop.

The `lw_dependent` test (Figures 12 and 13) exercises the `wb2` wakeup path. The diagram is compact, three instructions over seven cycles, and shows the `lw` occupying a single `M` cycle (cycle 4) between `I` and `C`, distinct from the `X/Y0--Y3` cells that ALU and MUL ops occupy. Instructions `b` (`add x4,x3,x6`, dependent on `lw`) and `c` (`add x5,x7,x8`, independent) both reach `I` on cycles 4 and 5 respectively, with no `i`-wait cycles for either: `b`'s wakeup arrives the same cycle the `lw` writes back through `wb2`, and `c`'s sources were never busy. The linetrace confirms tag 4 is a `lw` that renames `x03` to `p01`; tag 5 is a dependent `add` that reads `p01`; tag 6 is an independent `add` that does not. After the `lw` issues to MemUnit, the dmem response writes back through the third PRF port and the WB column prints `lw:p01`. In the same cycle, the IS column shows tag 5 dispatching (`5:add p06, p01, p33`), reading the just-written `p01` through the write-first bypass. Commit

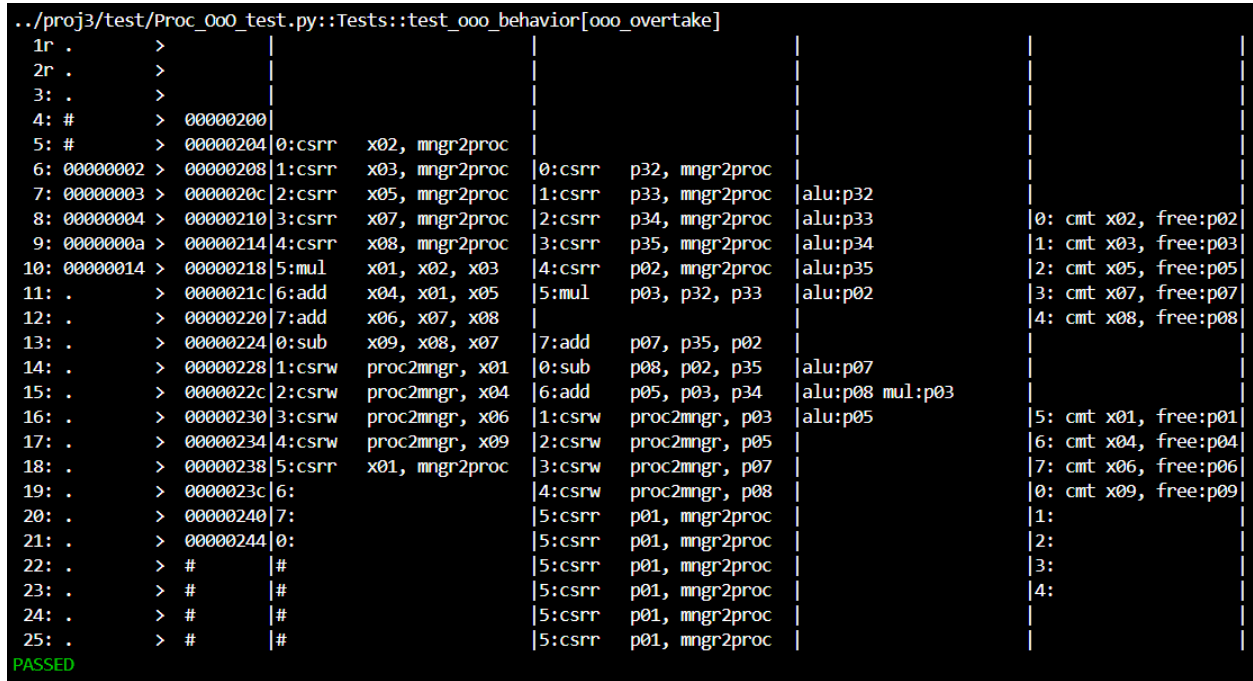


Figure 9: **overtake** linetrace. Independent operations (tag 7 add, tag 0 sub) issue and write back ahead of a dependent add (tag 6) waiting on a 4-cycle mul (tag 5). The WB column shows the alu:p08 mul:p03 dual writeback; the same cycle, tag 6 issues from the IQ on the bypass. Commit retires the four ROB tags in program order.

Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a: mul x3,x1,x2	F	D	I	Y0	Y1	Y2	Y3	C								
b: add x7,x4,x5		F	D	I	X	r	r	r	C							
c: add x3,x4,x1 (WAW x3)			F	D	I	X	r	r	r	C						
d: add x8,x3,x9 (reads new x3)				F	D	I	X	r	r	r	C					
e: add x11,x4,x1 (reads old x4)					F	D	I	X	r	r	r	C				
f: add x4,x9,x10 (WAR x4)						F	D	I	X	r	r	r	C			
g: add x13,x4,x1 (reads new x4)							F	D	I	X	r	r	r	C		
h: mul x6,x3,x2 (reads new x3)								F	D	I	Y0	Y1	Y2	Y3	C	
i: add x14,x9,x10									F	D	I	X	r	r	r	C

Figure 10: **waw_war_complex** pipeline diagram. Nine instructions over sixteen cycles. Instruction c (WAW on x3) reaches X one cycle before the older a's mul writes back; instruction e (reads old x4) reaches X one cycle before instruction f (WAR overwriter) reaches X. Both inversions are safe because rename assigns distinct physical registers per write. Trailing r runs are the PRF wait before in-order commit.

retires tags 4, 5, 6 in program order, with tag 4 freeing p03 (the old mapping of x03 before the 1w). Load completion plugs into the same renaming, wakeup, and commit machinery as ALU and MUL writeback; the only visible difference is the 1w: channel in the WB column and the M cell in the diagram.

4.4 Random Instruction-Mix Tests

At Professor Batten's suggestion that the two project 3 microarchitecture groups exchange artifacts, we obtained the random assembly test generator from the superscalar in-order group (Group 6: Edwin Chen,

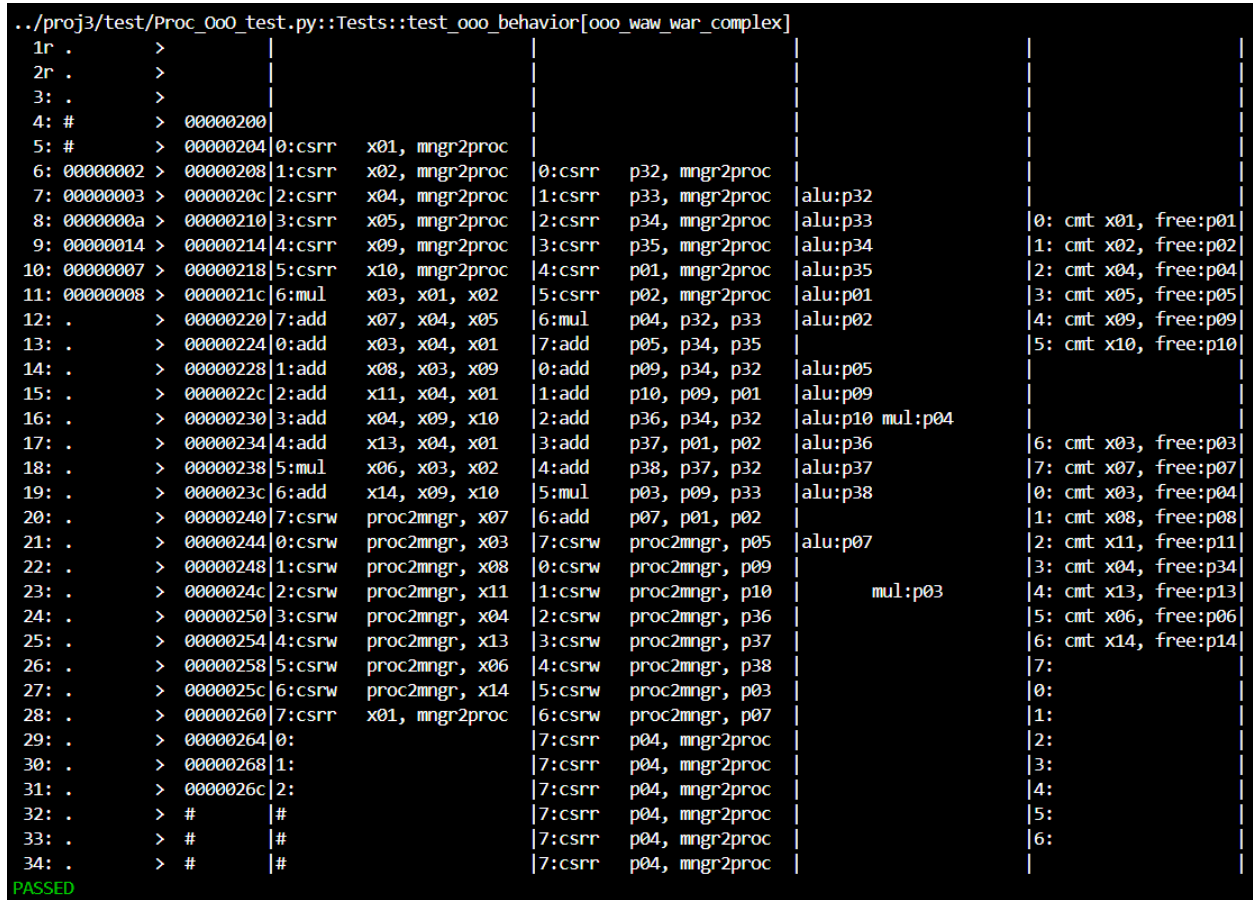


Figure 11: waw_war_complex linetrace. Multiple writes to x03 are renamed to distinct physical registers and committed in program order; each commit frees the previous paddr. mul:p03 late in the trace shows a freed paddr being recycled by a later allocation.

Instruction	1	2	3	4	5	6	7
a: lw x3, 0(x1)	F	D	I	M	C		
b: add x4, x3, x6 (depends on x3)		F	D	I	X	C	
c: add x5, x7, x8 (independent)			F	D	I	X	C

Figure 12: lw_dependent pipeline diagram. The lw (a) goes $F \rightarrow D \rightarrow I \rightarrow M \rightarrow C$ with M as the dmem-response cycle. The dependent add (b) reaches I the same cycle a's M produces lw:p01, waking up b through wb2 with no IQ-wait. The independent add (c) is never blocked.

Kian Mahmoodi, Wei Zheng) and adapted it to our processor. Their generator targets an ISA superset that includes branches, jal, jalr, lui, and auipc, none of which our single-issue OoO core implements. We pruned those opcodes from the op pools and from the per-category test factories, leaving four categories aligned with our supported ISA: **RR** (register-register arithmetic), **RIMM** (register-immediate arithmetic and shifts), **LW-mix** (loads interleaved with arithmetic), and **SW-mix** (stores with frequent readbacks, interleaved with loads and arithmetic). One adaptation was needed: their generator builds large constants with lui, which we lack, so the data-region base pointer 0x2000 is built from a sequence of addis on x31 before each memory-mix batch.

```

../proj3/test/Proc_OoO_test.py::Tests::test_ooo_behavior[ooo_lw_dependent]
1r .      > |
2r .      > |
3: .      > |
4: #      > 00000200 |
5: #      > 00000204 |0:csrr  x01, mngr2proc |
6: 00000200 > 00000208 |1:csrr  x06, mngr2proc |0:csrr  p32, mngr2proc |
7: 00000064 > 0000020c |2:csrr  x07, mngr2proc |1:csrr  p33, mngr2proc |alu:p32 |
8: 000000c8 > 00000210 |3:csrr  x08, mngr2proc |2:csrr  p34, mngr2proc |alu:p33 |0: cmt x01, free:p01 |
9: 0000012c > 00000214 |4:lw     x03, 0x000(x01) |3:csrr  p35, mngr2proc |alu:p34 |1: cmt x06, free:p06 |
10: .      > 00000218 |5:add    x04, x03, x06 |4:lw     p01, 0x000(p32) |alu:p35 |2: cmt x07, free:p07 |
11: .      > 0000021c |6:add    x05, x07, x08 |5:add    p06, p01, p33 |lw:p01 |3: cmt x08, free:p08 |
12: .      > 00000220 |7:csrw   proc2mngr, x05 |6:add    p07, p34, p35 |alu:p06 |4: cmt x03, free:p03 |
13: .      > 00000224 |0:csrw   proc2mngr, x03 |7:csrw   proc2mngr, p07 |alu:p07 |5: cmt x04, free:p04 |
14: .      > 00000228 |1:csrw   proc2mngr, x04 |0:csrw   proc2mngr, p01 |6: cmt x05, free:p05 |
15: .      > 0000022c |2:csrr   x01, mngr2proc |1:csrw   proc2mngr, p06 |7: |
16: .      > 00000230 |3: |2:csrr   p03, mngr2proc |0: |
17: .      > 00000234 |4: |2:csrr   p03, mngr2proc |1: |
18: .      > 00000238 |5: |2:csrr   p03, mngr2proc | |
19: .      > # |# |2:csrr   p03, mngr2proc | |
20: .      > # |# |2:csrr   p03, mngr2proc | |
21: .      > # |# |2:csrr   p03, mngr2proc | |
PASSED

```

Figure 13: `lw_dependent` linetrace. The `lw` at tag 4 writes back through the new `lw:p01` channel. The dependent `add` at tag 5 dispatches the same cycle through the `wb2` bypass. Commit retires tag 4 freeing the old `x03` paddr.

Differential testing against ProcFL. The framework runs each generated program twice. First it executes on ProcFL, our functional-level reference model, with all output suppressed; the final architectural register file values become the golden reference. The same body is then assembled into a self-checking pytest case that initializes registers, executes the random instruction stream, and ends with one `csrw proc2mngr, xr > val` per architectural register. This converts an open-ended “did the processor compute correctly?” question into a deterministic pass/fail check: any divergence in any committed register at the end of the program fails the test. Because the same program runs on ProcFL and ProcOoO, the comparison is a strict architectural-contract differential: the OoO design must not deviate from the in-order architectural state, no matter how aggressively rename and the IQ reorder execution internally. The reused directed tests already enforce this contract for hand-crafted programs; the random suite enforces it for sequences no human author would write.

Reproducibility and bisection. Each test factory reseeds Python’s RNG with a deterministic suffix of the form `ece6745-si-cat-b`, where *i* indexes the seed family, *cat* the category, and *b* the batch number. Failing tests reproduce byte-for-byte across runs and machines, and the parametrize table splits the workload into many small batches (50 instructions each) so a regression points at one tiny program rather than one giant one. We run three independent seed families (`s0`, `s1`, `s2`) for full regression and a single family for fast iteration during debugging. Both `NUM_BATCHES` and `BATCH_SIZE` are top-level parameters in `super_random.py`, so the regression scale can be dialed up for nightly runs or down for a quick smoke check without touching test logic.

Coverage. Table 3 summarizes the regression scale. At three seed families with 20 batches per category and 50 instructions per batch, the suite executes roughly $3 \times 20 \times 50 = 3,000$ random instructions *per category*, across 240 self-checking pytest cases in total. The mix exercises the structures most affected by OoO execution and not fully saturated by the directed suite: rename collisions on a small number of architectural registers (which forces frequent freelist allocate-free-allocate cycles), IQ wakeup with multiple producers in flight from a non-uniform op mix, ROB ordering under dual and triple writeback, and the asymmetric mem-vs-arith ordering rule when `lw` and `sw` are interleaved with arithmetic. The SW-mix category in particular issues a `lw` on the same address shortly after a `sw` with high probability, forcing the single-in-flight memory pipe

to serialize correctly under random spacing rather than under the hand-crafted spacing of `mem_order`. Re-running the same suite with delay injection enabled stresses `mem_issue_rdy` backpressure and IQ wakeup against arbitrary producer latencies, which no fixed-latency directed test reaches.

Category	Op pool	Cases
RR	add, sub, mul, and, or, xor, slt, sltu, sll, srl, sra	3×20
RIMM	addi, andi, ori, xori, slti, sltiu, slli, srli, srai	3×20
LW-mix	lw interleaved with random RR and RIMM ops over a 256-byte data region at 0x2000	3×20
SW-mix	sw (with frequent readback lw on the same address) interleaved with lw, RR, and RIMM ops	3×20
Total self-checking pytest cases		240

Table 3: Random instruction-mix regression. Each case runs one batch of 50 instructions whose final architectural register state is computed by `ProcFL` and checked on `ProcOoO`. Branches, `jal`, `jalr`, `lui`, and `auipc` are excluded because they are out of scope for this processor.

Outcome. The random suite found no new functional bugs on our final RTL, which is the outcome we hoped for: by the time we wired it in, the unit and directed layers had already caught the failure modes random programs typically expose. Its value going forward is as a regression net: any future change to rename, IQ, ROB, or the memory subsystem must continue to match `ProcFL` on thousands of unseen instruction sequences before it ships. Pruning the unsupported categories rather than disabling the suite entirely was deliberate: keeping the four categories we *do* support means the regression scales with our actual feature set, and re-enabling a category once branches and `jal/jalr` land is a one-line change in `ProcFL_super_random_test.py`.

4.5 Critical Bug and Lessons Learned

The most painful bug we hit involved back-to-back independent `mul` instructions. The IQ would issue any `mul` whose sources were ready, but the control unit was prepared for only one in-flight `mul` at a time. Two independent `mul`s in consecutive cycles caused the control unit’s view of the multiplier to diverge from its actual contents, producing wrong commit values and occasional hangs. The fix was to track the multiplier as a multi-stage pipelined resource inside the control unit.

The instructive part is that every reused single-instruction `mul` test passed. Each uses one `mul` surrounded by nops, so two simultaneously in-flight `mul`s never arise. Three improvements would have caught the bug sooner. First, a dedicated back-to-back `mul` directed test from day one; the `back_to_back_bypass` test in Section 4.3 was written *after* this bug. Second, a SystemVerilog assertion bounding the multiplier’s in-flight count to its pipeline depth, which would have fired on the first offending cycle rather than at the proc-level mismatch many cycles later. Third, extending the IQ unit test with a multi-cycle “functional unit busy” input, so the IQ/multiplier handshake could be checked in isolation. The broader lesson is that passing every single-instruction test gives a false sense of security on an OoO design: none of those tests puts more than one of the same long-latency instruction in flight at once. We applied this lesson when adding memory: the `ProcMemunit` unit test exercises the back-to-back case (`req_fire` and `resp_fire` on the same cycle), the IQ unit test exercises ALU traffic overtaking a stalled mem, and `mem_inflight` keeps the mem pipe saturated under random delays. The mem subsystem was covered against this class of bug from the first integration. The random instruction-mix suite of Section 4.4, layered on top of all of this, then provides a final differential check against `ProcFL` on sequences we never wrote by hand.

5 Evaluation

To isolate the contribution of OoO execution from that of integration with caches, we pushed four designs through the full ASIC flow at a **common 10 ns clock period**: the 5-stage in-order baseline processor (`ProcBase`), the OoO processor (`ProcOoO`), the baseline-cache system (`Base_SingleCoreSys`, “Sys-

Base”), and the OoO-cache system (OoO_SingleCoreSys, “SysOoO”). The common clock target lets us compare cycle counts directly without adjusting for cycle time. The four benchmarks (vmuladd2-unopt, chacha20-unrolled, dotprod-unopt-unrolled, vvadd-sw-opt) are the same across all four designs, with proc-level and system-level self-checking variants used as appropriate. Section 5.1 reports the standalone processor analysis, Section 5.2 the system-level analysis, Section 5.3 the cross-design performance comparison, Section 5.4 the comparison against the Group 6 superscalar, Section 5.5 the klayout views, and Section 5.6 the amoeba plot.

5.1 Standalone Processor: ProcBase vs. ProcOoO

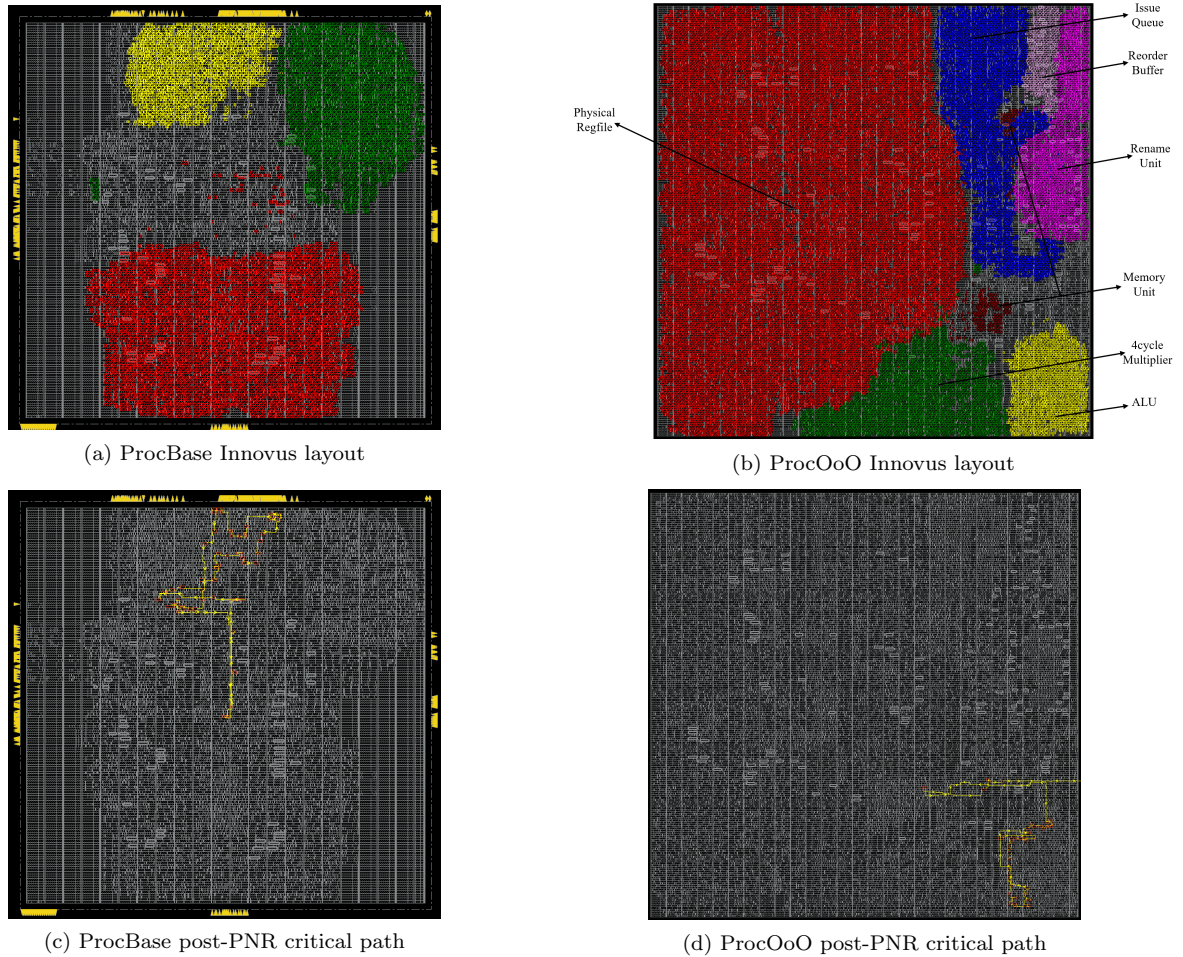


Figure 14: Standalone processor layouts and setup critical paths at the common 10 ns target.

ASIC flow summary. Table 4 summarizes the closure of both designs. Both pass the full flow (RTL, FF gate-level, back-annotated gate-level simulation, DRC, antenna, LVS) cleanly. ProcBase holds 0.17 ns of setup margin, ProcOoO 0.05 ns; both meet 10 ns, but the OoO design runs much tighter. ProcOoO’s post-PNR density is also significantly higher (76% vs. 51%), reflecting the denser register-file and queue structures introduced by renaming.

Area breakdown. Table 5 reports the post-filler area of the major blocks for both designs. ProcBase is dominated by the 2R1W architectural register file (38.3%), the 4-cycle multiplier (19.3%), and the ALU (10.8%); together these three components account for 68% of the core. ProcOoO’s 64-entry, 2R3W *physical* register file is so large (62.1%) that it dwarfs every other block: 4× the area of ProcBase’s RF alone,

Table 4: Standalone processor ASIC flow summary at 10 ns.

Item	ProcBase	ProcOoO
Synthesis setup slack (ns)	0.3478	0.0113
Post-PNR setup slack (ns)	0.1391	0.0330
Post-PNR hold slack (ns)	0.0104	0.0098
PrimeTime STA setup slack (ns)	0.1746	0.0494
PrimeTime STA hold slack (ns)	0.0077	0.0048
Post-PNR density (%)	50.73	76.29
Synthesis area (μm^2)	162,638	333,589
Post-PNR cell area (μm^2)	198,093	489,128
Post-PNR core area (μm^2)	406,832	667,552
DRC / Antenna / LVS	clean	clean

and larger than the entire baseline core. The OoO-specific structures (IQ, Rename Unit, ROB, MemUnit) collectively add another 18.6%. As a result, **ProcOoO** is 2.47 $\times$ the post-PNR cell area of **ProcBase**. The PRF, not the rename or commit logic, is the primary scalability bottleneck of this microarchitecture, and the area picture mirrors the power picture: PRF, IQ, multiplier, and rename also dominate dynamic power, while ROB, MemUnit, ALU, queues, and central control consume much less.

Table 5: Post-filler area breakdown of **ProcBase** and **ProcOoO**. Percentages are normalized to each design’s total post-PNR cell area. “—” indicates the structure does not exist in that design.

Module	ProcBase		ProcOoO	
	μm^2	%	μm^2	%
Register file (Base: 2R1W ARF; OoO: 2R3W PRF)	75,807	38.27	303,770	62.13
Integer multiplier	38,194	19.28	39,053	7.99
ALU	21,346	10.78	21,555	4.41
Issue queue	—	—	49,212	10.07
Rename unit	—	—	29,978	6.13
Reorder buffer	—	—	7,727	1.58
Memory unit	—	—	3,839	0.79
Control unit	6,544	3.30	2,268	0.46
Imem/Dmem queues	8,829	4.46	8,528	1.74
Other logic + pipeline registers	47,373	23.91	22,960	4.69
Total cell area	198,093	100	489,128	100

Critical path. Table 6 summarizes the PrimeTime setup critical paths reported by both designs. **ProcBase**’s path is the canonical fully-bypassed in-order signature: the X-stage operand register feeds the ALU, the ALU output enters the D-stage operand bypass mux, and the result loops back into the X-stage operand register within a single cycle. **ProcOoO**’s path is structurally different because there is no D-stage bypass network: the X-stage operand register feeds the ALU, but the ALU output is delivered to the dmem request output instead of looping back. In both cases the ALU combinational logic dominates the path (more than 70% of the data arrival time), confirming that **the OoO control structures (IQ, Rename, ROB) are not on the critical path**. The Phase 5 decision to merge Register Read into Issue is therefore vindicated, and additional frequency would have to come from ALU restructuring or carry-chain optimization.

Performance, power, and energy. Table 7 reports cycles, average power, and energy on the four standalone benchmarks. **ProcOoO** delivers cycle reductions ranging from $\sim 1\%$ (**chacha20-unrolled**, **vvadd-sw-opt**) to $1.51\times$ (**vmuladd2-unopt**); the benchmarks that supply ample independent ILP around long-latency multiplies (**vmuladd2-unopt**, **dotprod-unopt-unrolled**) show the largest reductions, while well-scheduled arithmetic kernels that already run near CPI=1 on the baseline (**chacha20-unrolled**, **vvadd-sw-opt**) leave noth-

Table 6: Setup critical-path summary for the standalone processor designs at 10 ns (PrimeTime STA).

	ProcBase	ProcOoO
Startpoint	dpath/op2_reg_X	dpath/op1_reg_X
Endpoint	dpath/op2_reg_X (bypass loop)	dmem_reqstream_msg
Data arrival	10.0864 ns	9.9006 ns
Setup slack	+0.1746 ns	+0.0494 ns
ALU contribution	~74%	84.36%
Path shape	ALU → bypass mux → op-reg loop	ALU → dmem request

ing for OoO to exploit. Section 5.3 discusses the attribution of these reductions to OoO execution versus to the pipelined multiplier. On the energy axis, **ProcOoO** loses on every benchmark by 1.7–2.3×: its average power is roughly 3× that of **ProcBase** (PRF, IQ, Rename, and the pipelined multiplier all switch continuously), and the cycle savings cannot offset this. The energy disadvantage is sharpest on **chacha20-unrolled**, where there is essentially no cycle savings to amortize the higher power against.

Table 7: Standalone processor cycles, power, and energy on the four benchmarks at 10 ns. Speedup is base cycles / OoO cycles; energy ratio is OoO energy / base energy.

Benchmark	ProcBase			ProcOoO			Ratio	
	Cyc.	P (mW)	E (nJ)	Cyc.	P (mW)	E (nJ)	Speedup	E ratio
vmuladd2-unopt	1916	13.5	258.66	1266	42.7	540.58	1.51×	2.09×
chacha20-unrolled	6785	27.6	1872.66	6724	63.2	4249.57	1.01×	2.27×
dotprod-unopt-unrolled	1014	12.6	127.76	815	27.4	223.31	1.24×	1.75×
vvadd-sw-opt	190	20.4	38.76	189	34.0	64.26	1.01×	1.66×

5.2 Processor-Cache System: SysBase vs. SysOoO

ASIC flow summary, and the timing inversion. Table 8 reports the system-level flow results. At 10 ns, **SysOoO** closes cleanly (PnR setup slack +0.0525 ns; STA +0.0743 ns), but **SysBase** **fails** to close after PnR (PnR setup slack −0.0018 ns; STA −0.0053 ns) despite passing synthesis with a comfortable +0.9071 ns slack. All four levels of simulation (PyMTL, VCS, FFGL, BAGL) pass for both designs; the failure is purely physical-timing. The critical-path analysis below explains why *the more complex* OoO design ends up easier to close than the simpler in-order baseline. Both designs share the same SRAM-macro signoff limitation: the top-level design does not appear in Calibre’s cell summary, so LVS and the DRC summaries are not generated; this is an integration limitation of the available flow, not a functional defect of either design.

Table 8: Processor-cache system ASIC flow summary at 10 ns. Negative setup slacks are highlighted in bold.

Item	SysBase	SysOoO
PyMTL RTL / VCS RTL / FFGL / BAGL simulation	all pass	all pass
Synthesis setup slack (ns)	0.9071	0.1780
Post-PNR setup slack (ns)	−0.0018	0.0525
Post-PNR hold slack (ns)	0.0472	0.0381
PrimeTime STA setup slack (ns)	−0.0053	0.0743
PrimeTime STA hold slack (ns)	0.0346	0.0291
Post-PNR density (%)	27.59	39.18
Post-PNR cell area (μm^2)	2,258,647	2,510,320
Post-PNR core area (μm^2)	3,830,876	3,830,876
DRC / LVS (SRAM signoff limit)	cell summary absent	cell summary absent

Critical path. Table 9 summarizes the two setup critical paths. The structural difference is decisive. **SysBase**’s path leaves the X-stage operand register, propagates through the ALU, and continues **across the**

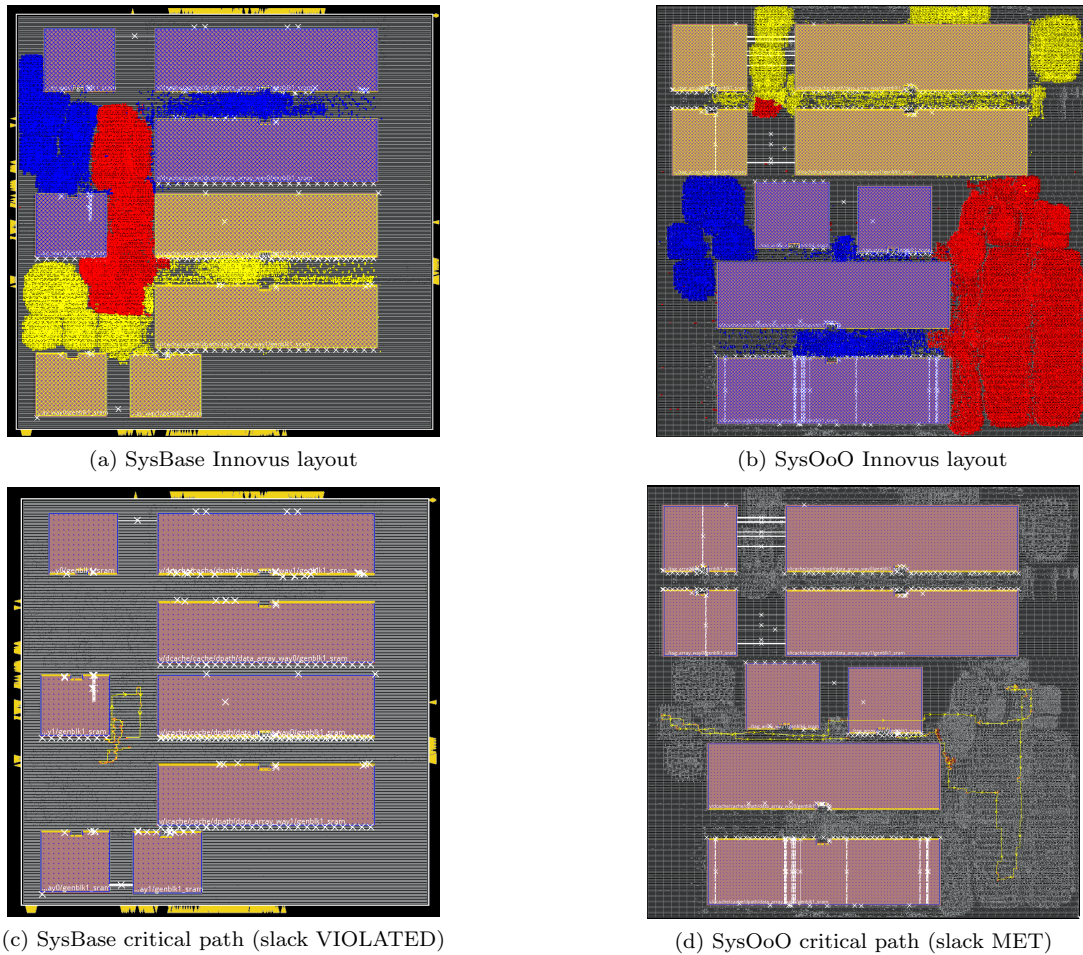


Figure 15: Processor-cache system layouts and setup critical paths at the common 10 ns target.

processor-cache boundary all the way into the I-cache data-array SRAM write-enable port: in a single cycle, the ALU output is required to settle the cache write control. **SysOoO**'s path also starts at the X-stage operand register and goes through the ALU, but it terminates at the **proc2mgr_msg** output port *within* the processor. The IQ stage in the OoO pipeline acts as a register barrier between dispatch and execute that prevents ALU-to-cache combinational paths from forming. As a consequence, the OoO core, despite its $2.5\times$ larger area and its extra IQ, Rename, and ROB structures, is the *easier* of the two to close at 10 ns at the system level.

Table 9: Setup critical-path summary for the system designs at 10 ns.

	SysBase	SysOoO
Startpoint	<code>proc/dpath/op2_reg_X</code>	<code>proc/dpath/op2_reg_X</code>
Endpoint	<code>icache/.../SRAM/WEN[12]</code>	<code>proc2mgr_msg[31]</code>
Path crosses	ALU $\rightarrow$ icache SRAM (cross-boundary)	ALU $\rightarrow$ proc2mgr (proc-internal)
Data arrival	10.4664 ns	9.8757 ns
Setup slack	-0.0053 ns	+0.0743 ns

Area breakdown. Table 10 reports the post-filler system area. The I- and D-caches together occupy about 80–90% of each system, and this fraction is roughly constant in absolute terms across the two designs ($\sim 2.04\text{ M } \mu\text{m}^2$ in SysBase, $\sim 2.00\text{ M } \mu\text{m}^2$ in SysOoO). The 11% difference in total system area is therefore

almost entirely the difference between the two cores: at the system level the OoO core takes 19.3% versus 8.5% for the baseline core, exactly the $2.5\times$ ratio observed in Section 5.1. Once SRAM-based caches are integrated, the core becomes a minority share of total area in both designs, and any further area-reduction effort would be more impactful on the cache subsystem than on the core.

Table 10: System area breakdown for **SysBase** and **SysOoO** at 10 ns. Percentages normalized to each system’s total post-PNR cell area.

Module	SysBase		SysOoO	
	μm^2	%	μm^2	%
Processor core	192,517	8.52	484,086	19.28
D-cache (SRAM + ctrl)	1,020,307	45.17	1,011,172	40.28
I-cache (SRAM + ctrl)	1,022,425	45.27	989,717	39.43
Other system glue	23,398	1.04	25,346	1.01
Total cell area	2,258,647	100	2,510,320	100

Power and energy. Table 11 reports the four-benchmark cycle, power, and energy comparison at the system level. The cycle results match Section 5.3 (Table 13); we focus here on the energy trend, which is qualitatively different from the standalone picture. At the system level, the SRAM-based caches dominate dynamic-power dissipation, which moderates the proc-side power gap between OoO and baseline. **SysOoO** draws $1.1\text{--}1.4\times$ more average power than **SysBase**, but the cycle savings of OoO are now enough to *erase* the energy disadvantage on three of the four benchmarks: **SysOoO** consumes 1–2% less energy than **SysBase** on **vmuladd2-unopt**, **dotprod-unopt-unrolled**, and **vvadd-sw-opt**. The only loss is **chacha20-unrolled**, where OoO supplies essentially zero cycle savings and so its higher power translates directly into 17% higher energy. Once integrated with caches, the OoO core is a near-energy-neutral upgrade over the in-order baseline on the benchmarks that supply ILP, while remaining a clear performance win whenever the program has independent work to overlap.

Table 11: System-level cycles, average power, and energy at 10 ns.

Benchmark	SysBase @ 10 ns			SysOoO @ 10 ns			Ratio	
	Cyc.	P (mW)	E (nJ)	Cyc.	P (mW)	E (nJ)	Speedup	E ratio
vmuladd2-unopt	6182	81.6	5044.51	4303	116.1	4995.78	$1.44\times$	$0.99\times$
chacha20-unrolled	19022	95.0	18070.90	18830	112.1	21108.43	$1.01\times$	$1.17\times$
dotprod-unopt-unrolled	4029	92.5	3726.83	3463	105.5	3653.47	$1.16\times$	$0.98\times$
vvadd-sw-opt	776	118.2	917.23	687	131.0	899.97	$1.13\times$	$0.98\times$

Where the system power goes. The new `run.log` flow reports only total system power, but the distribution can be inferred unambiguously from the area picture and the dynamic activity of each block. The SRAM-based I- and D-caches together occupy 79.7% of the SysOoO system area (Table 10), and SRAM macros switch every fetch and every load/store cycle; the processor core, at 19.3% of area, contains the PRF, IQ, Rename, ROB, multiplier and ALU, all of which switch on every issue and writeback. Combining these dynamic patterns with the area weights places the cache subsystem at clearly above 70% of system dynamic power on every benchmark in Table 11, and the processor core in the minority. This is the inverse of the standalone picture, where the PRF/IQ/MUL/ Rename structures dominate. The benchmark-to-benchmark spread in system power (105.5–131.0 mW in SysOoO, 81.6–118.2 mW in SysBase) is largely driven by D-cache activity: **vvadd-sw-opt** has the densest store traffic per cycle and is the highest-power benchmark in both designs.

5.3 Cross-Design Performance Comparison

This section combines the cycle results of Sections 5.1 and 5.2 into a single side-by-side performance comparison and discusses the sources of the OoO speedup on a per-benchmark basis.

Standalone cycle comparison. Table 12 reports cycle counts, instruction counts, CPI, and speedup for the standalone designs. The 5-stage baseline has a fully-bypassed in-order pipeline; the only sources of extra cycles beyond CPI=1 are the multi-cycle multiplier (`vmuladd2-unopt`, `dotprod-unopt-unrolled`) and load-use hazards. The OoO design runs the same kernels at CPI very close to 1 on all four benchmarks; the speedup over baseline is therefore essentially the elimination of multiplier and load-use stalls.

Table 12: Standalone processor performance comparison at 10 ns. Instruction counts are identical between the two designs because both execute the same proc-level program.

Benchmark	ProcBase			ProcOoO			Speedup
	Cycles	Insts	CPI	Cycles	Insts	CPI	
<code>vmuladd2-unopt</code>	1916	1205	1.59	1266	1205	1.05	1.51×
<code>chacha20-unrolled</code>	6785	6717	1.01	6724	6717	1.00	1.01×
<code>dotprod-unopt-unrolled</code>	1014	705	1.44	815	705	1.16	1.24×
<code>vvadd-sw-opt</code>	190	182	1.04	189	182	1.04	1.01×

Multiplier microarchitecture as a confound. An important caveat applies to the standalone numbers above: **the two designs use different multiplier microarchitectures.** The baseline integer multiplier is a 4-cycle *iterative* unit that cannot accept a new `mul` until the previous one has completed (effective throughput one `mul` per 4 cycles), while the OoO multiplier is a fully *pipelined* 4-cycle unit that can accept a new `mul` every cycle (throughput one `mul` per cycle). On the two multiplier-heavy benchmarks (`vmuladd2-unopt` contains two `muls` per element with a back-to-back RAW chain; `dotprod-unopt-unrolled` contains one `mul` per element), part of the observed speedup is attributable to the higher multiplier throughput rather than to OoO scheduling per se. The two no-`mul` benchmarks (`chacha20-unrolled`, `vvadd-sw-opt`) therefore isolate the OoO-only contribution and both show $\sim 1.01\times$ speedup, indicating that on already-tightly-scheduled arithmetic kernels OoO buys very little. We interpret the speedup on `vmuladd2-unopt` and `dotprod-unopt-unrolled` as the *joint* contribution of (i) pipelined-multiplier throughput and (ii) OoO's ability to overlap the 4-cycle `mul` latency with independent younger instructions; the joint contribution cannot be fully separated without a controlled experiment that swaps the multiplier between the two pipelines.

System cycle comparison. Table 13 reports the same comparison at the system level, using cycle counts from the PyMTL self-checking pytest harness (which also produces the cache-miss-rate statistics). Both designs service the same access streams, so the cache miss rates are identical between SysBase and SysOoO: I-cache misses every 4th fetch (the benchmarks are fully unrolled and branch-less; with a 16-byte I-cache block holding four 32-bit instructions, the steady-state fetch pattern is miss-hit-hit-hit), and the D-cache miss rate is driven entirely by each kernel's data access pattern.

Table 13: System-level performance and cache statistics at 10 ns. Cycle counts and CPI are from PyMTL self-checking pytest (which produces the cache statistics); the small offset ($<0.1\%$) versus Table 11 run.log cycles is the test-harness prologue/epilogue.

Benchmark	SysBase		SysOoO		I\$ MR	D\$ MR	Speedup
	Cyc.	CPI	Cyc.	CPI			
<code>vmuladd2-unopt</code>	6179	4.39	4300	3.05	0.25	0.75	1.44×
<code>chacha20-unrolled</code>	19019	2.78	18827	2.75	0.25	0.13	1.01×
<code>dotprod-unopt-unrolled</code>	4026	4.44	3460	3.82	0.25	0.75	1.16×
<code>vvadd-sw-opt</code>	773	3.13	684	2.78	0.25	0.25	1.13×

The system speedups track the standalone speedups closely ($\sim 1.01\times$ on **chacha20-unrolled**, $1.13\text{--}1.16\times$ on **vvadd-sw-opt** and **dotprod-unopt-unrolled**, $1.44\times$ on **vmuladd2-unopt**), with two notable differences: (1) The standalone $\sim 1.01\times$ on **vvadd-sw-opt** becomes $1.13\times$ at the system level, because the D-cache miss rate of 0.25 injects gaps into the issue stream that OoO can fill with independent younger work while the in-order baseline stalls. (2) The standalone $1.51\times$ on **vmuladd2-unopt** drops to $1.44\times$ at the system level: the D-cache miss rate of 0.75 makes memory latency the dominant component of execution time, leaving less room for the OoO core and the pipelined multiplier to differentiate. The same mechanism applies to **dotprod-unopt-unrolled** ($1.24\times$ standalone $\rightarrow 1.16\times$ system). The system-level message matches the standalone one: dynamic scheduling pays off when the program supplies independent work to overlap, and once cache misses dominate execution time, the ceiling on what OoO can recover is correspondingly lower.

5.4 Performance Comparison Between Out-of-Order and Superscalar

To further evaluate the effectiveness of the OoO processor, we compare its performance against the in-order superscalar processor developed by Group 6. The Group 6 processor is a dual-issue in-order design, while our processor uses single-issue out-of-order execution. To make the comparison fair, both processors use a 4-cycle pipelined integer multiplier, and the same four benchmarks are used for evaluation. Since our current processor does not support control-flow instructions, all benchmarks are implemented in an unrolled form without branches or jumps. The OoO cycle counts in this section refer to the standalone OoO processor under the same 10 ns target as Section 5.1.

The selected benchmarks cover two different types of instruction behavior. The first two (**chacha20** and **vvadd-sw-opt**) are relatively favorable to in-order execution because they contain abundant adjacent ILP that a dual-issue processor can exploit. In contrast, **vmuladd2-unopt** and **dotprod-unopt** contain stronger data dependences: each loop body has two consecutive RAW dependences where a multiply instruction acts as the producer. These dependent instruction chains can repeatedly block an in-order pipeline, causing stalls. These benchmarks therefore show whether out-of-order issue can tolerate long-latency producer instructions by allowing independent ready instructions to make progress.

Table 14: Performance comparison between our OoO and the Group 6 superscalar processor.

Benchmark	OoO (single-issue)			Superscalar (dual-issue in-order)			Speedup
	Cycles	Insts	CPI	Cycles	Insts	CPI	
chacha20	6724	6717	1.00	3541	6713	0.53	$0.53\times$
vvadd-sw-opt	189	182	1.04	107	178	0.60	$0.57\times$
vmuladd2-unopt	1266	1205	1.05	1801	1201	1.50	$1.42\times$
dotprod-unopt	815	705	1.16	801	701	1.14	$0.98\times$

On the two ILP-rich benchmarks, the dual-issue superscalar wins decisively (CPI 0.53 and 0.60 vs. OoO 1.00 and 1.04), because a single-issue OoO core cannot exceed one instruction committed per cycle regardless of how aggressively it reorders. On **vmuladd2-unopt**, the OoO design recovers a $1.42\times$ advantage by hiding the multiply-RAW chains that block the in-order issue. On **dotprod-unopt**, the two designs are within 2% of each other. The pattern is the canonical contrast between issue-width and scheduling-window scaling: superscalar buys throughput on parallelizable code, OoO buys latency-tolerance on serial code.

Table 15: Area comparison between our OoO and the Group 6 superscalar.

Processor	Synth Area (μm^2)	PNR Cell Area (μm^2)
OoO Processor (single-issue)	333,589	489,128
In-order Superscalar (dual-issue)	242,745	328,044

The OoO processor has a larger area than the Group 6 in-order superscalar processor. This is primarily caused by the extra storage and bookkeeping required by out-of-order execution: a 64-entry physical register file, plus an issue queue, rename unit, and reorder buffer, together account for $\sim 18\%$ of the OoO core area (Table 5). The superscalar processor uses a conventional 32-entry architectural register file (with extra ports for dual-issue) and no rename or ROB structures.

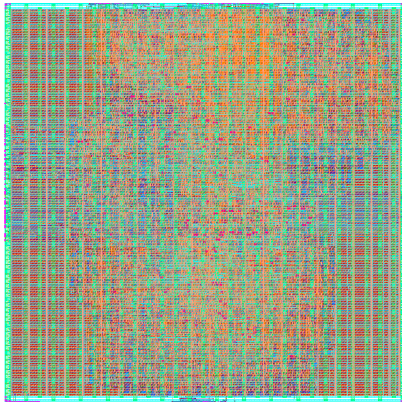
Table 16: Power and energy comparison between our OoO and the Group 6 superscalar.

Benchmark	OoO		In-order Superscalar	
	Power (mW)	Energy (nJ)	Power (mW)	Energy (nJ)
chacha20	63.2	4249.57	50.0	1770.51
vvadd-sw-opt	34.0	64.26	32.1	34.35
vmuladd2-unopt	42.7	540.58	17.8	320.58
dotprod-unopt	27.4	223.31	17.8	142.58

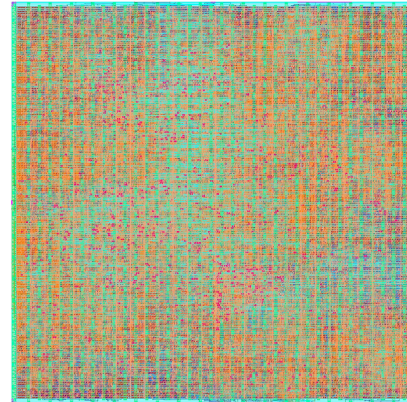
Although our OoO processor is single-issue, its datapath contains parallel execution units (ALU, pipelined multiplier, memory unit), and the IQ/PRF/Rename/ROB structures all switch every cycle. The total energy follows directly: on **chacha20** and **vvadd-sw-opt** the superscalar is both faster and lower-power and so has much lower energy; on **vmuladd2-unopt** the OoO core achieves the better performance but the OoO hardware overhead keeps its energy higher; on **dotprod-unopt** the cycle counts are close and the energy gap comes from the higher OoO average power.

5.5 Klayout Views of the Four Designs

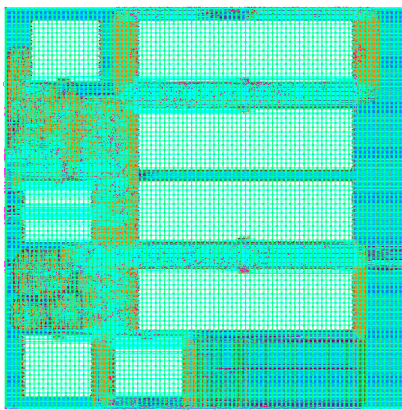
Figure 16 shows the final klayout streams for all four designs at the same 10 ns target. The standalone processors sit on a $\sim 667 \mu\text{m}$ core, the systems on a $\sim 1,957 \mu\text{m}$ core; the cache SRAM macros and processor block colors in the system images correspond directly to the placement seen in Figures 14 and 15.



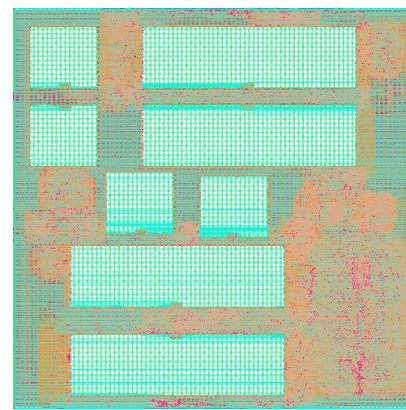
(a) ProcBase



(b) ProcOoO



(c) SysBase



(d) SysOoO

Figure 16: Klayout views of the four post-layout designs at the common 10 ns target.

5.6 Amoeba Plot

Figure 17 summarizes the four-design comparison on a five-axis amoeba plot. Each axis represents one normalized metric in “larger-is-better” form: 1/Cycle Time, 1/Avg. Cycles, 1/Area, 1/Avg. Power, and 1/Avg. Energy. The averages are taken over the four benchmarks. On each axis the **better of the two designs** is normalized to 1.0 and the other is plotted as its ratio, so both polygons fit inside the unit pentagon and the side that bulges *outward* on each axis is the winner there. The standalone amoeba in Figure 17(a) shows the classic in-order vs. OoO tradeoff: **ProcBase** dominates Area, Power, and Energy by a wide margin; **ProcOoO** wins only on Cycles. The system amoeba in Figure 17(b) is the more important picture: once the cache subsystem is included, the four non-cycle metrics rebalance to within 10–17% of each other, **SysOoO** is essentially energy-neutral against **SysBase**, and the cycle advantage remains. The underlying numerical data is given in Table 17.

Table 17: Amoeba plot input data. Raw values are averaged over the four benchmarks; normalized rows are radial coordinates for Figure 17, computed so that the better design on each axis (i.e. the larger value of 1/metric) maps to 1.0.

Metric / Axis	Standalone		System	
	Base	OoO	Base	OoO
Cycle Time (ns)	10.00	10.00	10.00	10.00
Avg. Cycles	2,476	2,249	7,499	6,818
Area ($\text{k}\mu\text{m}^2$)	198	489	2,259	2,510
Avg. Power (mW)	18.5	41.8	96.83	116.18
Avg. Energy (nJ)	574.5	1,269.4	6,939.9	7,664.4
Normalized 1/CycleTime	1.000	1.000	1.000	1.000
Normalized 1/Cycles	0.908	1.000	0.909	1.000
Normalized 1/Area	1.000	0.405	1.000	0.900
Normalized 1/Power	1.000	0.443	1.000	0.834
Normalized 1/Energy	1.000	0.453	1.000	0.906

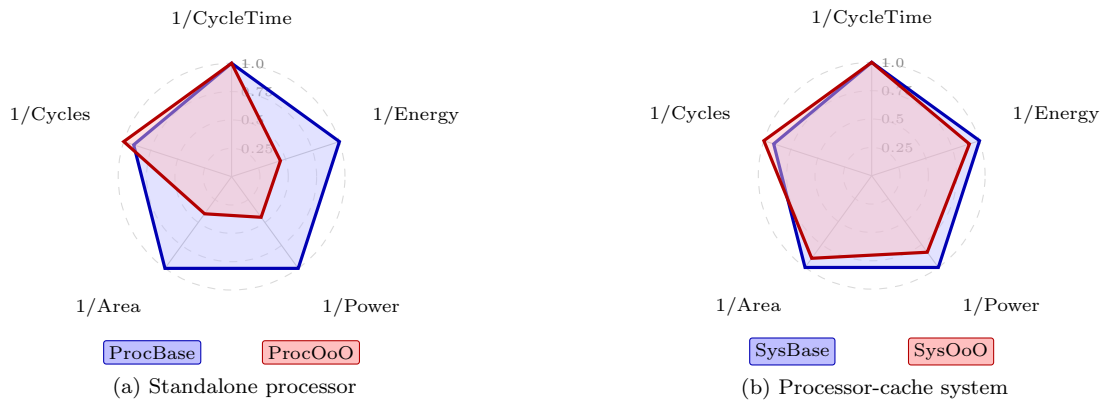


Figure 17: Five-axis amoeba plots for the standalone (a) and system (b) comparisons. On each axis the better-of-the-two design maps to 1.0, so the polygon that reaches the outer ring on an axis is the winner for that metric. Standalone: OoO wins on Cycles but loses by a wide margin on Area, Power, and Energy. System: the four non-cycle metrics tighten to within 10–17%, while the cycle advantage of OoO is preserved.

References

- [1] R. E. Kessler, “The Alpha 21264 Microprocessor,” *IEEE Micro*, vol. 19, no. 2, pp. 24-36, March-April 1999.

- [2] K. C. Yeager, "The MIPS R10000 Superscalar Microprocessor," *IEEE Micro*, vol. 16, no. 2, pp. 28-41, April 1996.